Journal of Cybersecurity Education, Research and Practice

Manuscript 1220

The Impact of AI Use in Programming Courses on Critical Thinking Skills

Christian Jay St Francis Clarke

Abdullah Konak

The Impact of AI Use in Programming Courses on Critical Thinking Skills

Abstract

Proficiency in computer programming extends far beyond memorizing syntax; it depends on the cultivation of critical thinking. Computer programming requires multiple interconnected competencies, including systematic problem analysis, algorithmic reasoning, mastery of programming languages, debugging capabilities, a comprehensive understanding of software development methodologies, rigorous testing practices, and systematic troubleshooting approaches. These skills are also essential for cybersecurity experts; cybersecurity programs require several programming courses to enhance students' technical and critical thinking skills. Generative AI (GenAI) technologies have fundamentally changed the process of developing applications and approaches to teaching coding. The growing use of GenAI technologies by students in writing computer code has raised not only concerns about academic integrity but also their impact on the development of critical thinking skills, which are typically reinforced by assigning coding exercises that require students to rely on those skills to solve them. This paper presents the results of an empirical study examining students' perceptions of using GenAI in coding assignments and its impact on critical thinking among cybersecurity students. The findings show that students use GenAI extensively to debug and correct their code. Findings also show that the use of GenAI should be structured to fully support the development of critical thinking skills.

Keywords

critical thinking skills, AI, computer education, cybersecurity

The Impact of AI Use in Programming Courses on Critical Thinking Skills

Christian Jay St Francis Clarke
Information Sciences and Technology
Penn State Berks
Reading, PA, USA
cjc6362@psu.edu

Abdullah Konak
Information Sciences and Technology
Penn State Berks
Reading, PA, USA
ORCID: 0000-0001-6250-7825

***Abstract*—Proficiency in computer programming extends far beyond memorizing syntax; it depends on the cultivation of critical thinking. Computer programming requires multiple interconnected competencies, including systematic problem analysis, algorithmic reasoning, mastery of programming languages, debugging capabilities, a comprehensive understanding of software development methodologies, rigorous testing practices, and systematic troubleshooting approaches. These skills are also essential for cybersecurity experts; cybersecurity programs require several programming courses to enhance students' technical and critical thinking skills. Generative AI (GenAI) technologies have fundamentally changed the process of developing applications and approaches to teaching coding. The growing use of GenAI technologies by students in writing computer code has raised not only concerns about academic integrity but also their impact on the development of critical thinking skills, which are typically reinforced by assigning coding exercises that require students to rely on those skills to solve them. This paper presents the results of an empirical study examining students' perceptions of using GenAI in coding assignments and its impact on critical thinking among cybersecurity students. The findings show that students use GenAI extensively to debug and correct their code. Findings also show that the use of GenAI should be structured to fully support the development of critical thinking skills.**

Keywords— Computer programming, critical thinking, algorithmic reasoning, cybersecurity, Generative AI (GenAI), teaching coding, academic integrity, AI in education, computational thinking.

I. Introduction

Generative AI (GenAI) technologies like ChatGPT are transforming computing classes by offering students new ways to approach tasks such as debugging, code generation, and understanding complex programming concepts [1]. Experts using AI coding report that their development cycle now involves mainly prompting and editing, with 80% of code generated by AI, according to a recent article published by Forbes [2]. These tools can simplify technical challenges, provide personalized assistance, and help students accelerate their learning process. Instructors are increasingly integrating GenAI into programming classes, using it to supplement traditional teaching methods. For example, they might assign students to use GenAI to refine code after initial drafting or to explore alternative coding solutions. Instructors also employ GenAI to facilitate peer learning, where students compare AI-generated solutions with their code, prompting discussions around optimization and best practices [3]. By utilizing GenAI in this structured way, instructors help students develop coding proficiency while adapting to the latest advancements in AI-assisted learning. However, the impact of GenAI on critical thinking, particularly in programming education, is a growing concern among educators.

Critical thinking involves dissecting problems, understanding their underlying structure, and formulating creative solutions [4]. According to a comprehensive Delphi Study conducted by the American Philosophical Association [5], critical thinking is purposeful and self-regulatory, involving interpretation, analysis, and evaluation of evidence and context. These skills are essential for an innovation mindset [6]. The Socratic dialogue, known for stimulating deep thought through questioning, has long been a core teaching method in computer science education [7]. In the context of programming, students typically engage in discussions, challenging one another's ideas and approaches to problem-solving. In programming, this means not only writing functional code but also questioning assumptions, weighing evidence, and reaching logical conclusions. Programming inherently fosters critical thinking through the process of problem-solving. When students engage in developing algorithms or debugging errors, they continually apply analytical reasoning and refine their approaches. When paired with GenAI, there is an opportunity for Socratic dialogue to flourish, as GenAI can encourage students to question and refine their solutions [8]. This potential must be balanced with the risk of reducing students' independent critical thinking by providing too many pre-generated answers. With the rise of GenAI, there is growing concern among faculty that students may rely too heavily on these tools, potentially bypassing the critical cognitive processes required for true mastery of subjects [9].

Critical thinking skills and knowledge are generally considered in six dimensions: interpretation, analysis, evaluation, inference, explanation, and self-regulation [5]. Each of these skills plays a crucial role in problem-solving. In programming, interpretation involves understanding and processing information, identifying key concepts, and

recognizing patterns that guide logical flow and coding strategies [10]. Analysis enables programmers to break down complex concepts or problems into smaller pieces and examine how individual components interact within the program's structure. Evaluation furthers this process by assessing the credibility, efficiency, and suitability of solutions, such as determining the optimal algorithm for a specific task based on its complexity and runtime requirements. Inference builds on these skills by allowing programmers to draw logical conclusions, such as predicting program behavior from input patterns or diagnosing issues during debugging [11]. The ability to clearly articulate the rationale behind coding decisions requires explanation, making reasoning accessible to both self and peers, a critical component of collaborative programming environments [12]. Finally, self-regulation involves monitoring and adjusting one's approach. This allows students to reflect on and refine their code in alignment with best practices, ensuring continuous improvement and adaptability [13]. These critical thinking skills are equally indispensable in the field of cybersecurity [14], where they emphasize key tasks such as threat detection, vulnerability analysis, and incident response. For instance, interpretation aids in deciphering complex data logs and identifying patterns indicating potential threats. At the same time, analysis and evaluation support the design and implementation of robust defense mechanisms tailored to specific attack vectors [15]. Furthermore, inference enables security professionals to anticipate adversarial behavior and predict potential exploit paths. At the same time, a clear explanation facilitates the communication of findings and solutions to stakeholders, which is crucial during incident resolution. Finally, self-regulation promotes a proactive approach to adapting to the evolving threat landscape, ensuring the continuous refinement of security measures and strategies [16].

This paper aims to investigate the role of GenAI in enhancing or hindering critical thinking in programming classes with the primary research question: "How does the use of GenAI in programming courses impact students' perceived development of critical thinking skills?" Specifically, this paper explores students' perceptions of how these tools influence the key components of critical thinking. Through both qualitative and quantitative analysis, our study assesses students' self-perception about whether GenAI helps them deepen their understanding or fosters overreliance, ultimately impeding their development of essential problem-solving skills. Secondly, the study aims to understand how students use GenAI in coding tasks. Understanding student perceptions is crucial for identifying both opportunities and challenges in integrating GenAI into programming education and cybersecurity education, which is the primary contribution of this paper.

II. Literature Review

A. Integrating Socratic Dialogue-Based Learning to Cultivate Critical Thinking

Computer science classes are not merely about mastering programming languages; it is about nurturing problem-solvers, analytical thinkers, and effective communicators [17]. In this pursuit, the infusion of Socratic dialogue-based learning into code comprehension offers a promising avenue to cultivate critical thinking skills among computer science students [18]. Socratic dialogue, rooted in questioning, reasoning, and discussion, aligns seamlessly with the workings of code comprehension. By integrating this methodology into computer science education, students are encouraged to engage in an interactive exploration of source code. This approach shifts the process of understanding code from a passive activity to an engaging dialogue-driven experience.

Auto-generated Socratic dialogue tools, such as ChatGPT, can act as catalysts in this learning process, facilitating structured interactions between students and code. These tools generate customized questions, guiding students to explore code functionalities, predict outputs, analyze logical sequences, and propose alternative solutions. By engaging in these dialogues, students not only decipher code but also enhance their critical thinking skills. The impact of integrating Socratic dialogue into computer science education is multifaceted. It improves problem-solving abilities by encouraging students to view code as a puzzle to be solved. Through dissecting code snippets, identifying patterns, and articulating solutions, students develop agility in problem-solving—a core competency in computer science [19]. Secondly, this approach refines analytical thinking. Students engaging in dialogue-driven code exploration learn to scrutinize code structures, discern underlying logic, and spot potential errors or optimization opportunities. Such examinations can nurture a mindset for critical analysis. Moreover, the practice of articulating interpretations within a dialogue format refines reasoning and communication skills. Defending viewpoints, engaging in peer discussions, and justifying solutions foster a culture of constructive discourse, enhancing not only technical understanding but also the ability to convey ideas effectively [20].

Implementing Socratic dialogue-based learning requires a shift in pedagogy towards interactive and collaborative learning environments. Educators can integrate these tools into coding exercises, projects, or group discussions, fostering a culture of inquiry and exploration. Assessment strategies should encompass not only measuring code comprehension but also evaluating the depth of reasoning displayed in dialogues and observing improvements in problem-solving approaches. Challenges exist, such as adapting tools to diverse learning styles and ensuring inclusivity, but tackling these challenges could pave the way for innovative advancements in computer science education. The integration of Socratic dialogue-based learning signals a new era in computer science education. It's not just about coding; it's about nurturing agile minds that possess essential skills, such as critical thinking, problem-solving, and effective communication, in a dynamic technological landscape. This approach underscores the evolution from rote learning to holistic skill development, empowering students to innovate and adapt to the demands of the digital age [21].

B. Frameworks for the Socratic Method

The Socratic method can be a model of critical thinking through intellectual inquiry [22]. The exploration of formal models governing the Socratic method prompts an examination of how computational methods intersect with knowledge engineering and the timeless principles that underpin Socratic

questioning. The Socratic method, rooted in dialectical questioning and reasoned discourse, transcends conventional teaching methodologies [23]. It propels learners to examine concepts, challenge assumptions, and gain heightened insights through structured inquiry, which is a cornerstone of cognitive engagement [23].

Huse and Le [24] present three-phase computational models of Socratic dialogues with a chatbot (called Tx), searching for examples, searching for attributes, and generalizing attributes. In the first phase, the chatbot tasks participants to provide examples and then requests clarification from the participant who gave the example. Other participants can raise concerns, prompting further explanation from Tx. If no problems arise, the example is added to a collection. Once enough examples are gathered, participants vote on the best one for further discussion. In the second phase, the chatbot uses the best-voted example to elicit attributes. Participants suggest attributes, and Tx explains them. Other participants can ask questions or give opinions, leading to further clarification or modification of the attributes. This process continues until a sufficient number of attributes have been collected. In the third phase, the chatbot guides participants in generalizing the attributes. Tx selects attributes to generalize and explains the reasoning. Other participants can ask questions or give opinions, leading to further discussion and refinement. The process continues until the list of generalized attributes is deemed sufficient. The convergence of such computational models with the Socratic method bears profound implications for education and computer science by leading to intelligent tutoring systems, adaptive learning platforms, or dialogue-based interfaces [25]. Such tools hold promise in nurturing critical thinking, tailoring learning experiences, and advancing knowledge acquisition across domains in the digital era [23].

C. Use of AI and Critical Thinking

The use of AI in education is an emerging and growing area of research. Recent research on the impact of AI on critical thinking has typically reported positive outcomes, particularly in the use of chatbots in education. A systematic review by Premkumar, Yatigammana, and Kannangara [26] examined the impact of GenAI on the critical thinking skills of undergraduates. The study found that about half of the reviewed papers suggest that GenAI can enhance critical thinking skills. For example, in a recent study focusing on ChatGPT's impact on education, Suriano et al. [27] examined how student interactions with AI chatbots influence the development of critical thinking. Their research, conducted with 213 Italian students, revealed significant correlations between students' attitudes toward AI, trust, engagement levels, and critical thinking performance. The authors reported that engagement with AI had a more substantial impact on critical thinking than knowledge acquisition alone. While supporting AI chatbots as valuable educational tools, Suriano et al. (2025) emphasize the need for pedagogical approaches that promote active engagement and deep understanding rather than passive consumption of AI-generated information. Nassar [28] explored the future of AI and its reliance on critical thinking. Nassar [28] emphasizes that AI's development is intertwined with human cognitive skills, which enhance its complexity and capabilities. Muthmainnah et al. [29] examined the impact of AI use on English foreign language learners. Using mixed methods, the research found that AI friend apps enhance students' critical thinking skills, trust, self-confidence, open-mindedness, and maturity.

Styve et al. [30] investigated the integration of GenAI tools, such as GitHub Copilot and ChatGPT, in an introductory programming course to enhance critical thinking practices. Their study highlights concerns about novice programmers' reliance on AI without critical reflection. Through surveys conducted before and after AI-based assignments, the study found that students' awareness of AI's possibilities and limitations, as well as their critical thinking skills, improved. Yilmaz and Yilmaz [3] explored the impact of using ChatGPT in programming education. They found that students who used ChatGPT showed significant improvements in computational thinking skills, programming self-efficacy, and motivation compared to those who did not use the tool. Ododo et al. [31] investigated the impact of using ChatGPT in Java programming courses and found that students who utilized ChatGPT for coding assistance demonstrated significant improvements in computational thinking skills and programming self-efficacy compared to their peers. Silva et al. [32] discussed the challenges and benefits of using ChatGPT in higher education, particularly in software programming. The study highlighted several advantages of ChatGPT, such as rapid response generation, high accuracy, and its potential to support sustainable coding practices. However, they also pointed out the risk of students becoming overly dependent on AI-generated code, which could hinder their understanding of fundamental concepts.

D. Integration of ChatGPT into Programming Courses

The integration of GenAI into programming education presents a significant opportunity to enhance critical thinking skills among computer science majors. This intersection of critical thinking and GenAI in computer science education is particularly relevant as educators aim to prepare students for a rapidly evolving industry. GenAI can enable students to explore diverse coding paradigms, analyze errors, and reflect on their problem-solving approaches in dynamic ways [33]. Integrating GenAI into programming courses can significantly enhance the learning process, particularly in areas such as debugging, code revision, and fostering critical thinking. Hashmi et al. [34] highlight that GenAI tools can assist students in identifying and resolving errors in their code, providing insights into error messages, suggesting possible fixes, and explaining the logic behind the errors. This support helps students understand the root cause of their issues while gaining deeper insights into debugging practices. For debugging, this enables students to identify the root cause of problems while gaining a deeper understanding of debugging practices [35]. Additionally, it can provide guidance on common pitfalls and programming misconceptions, helping students avoid recurring mistakes and enhance their problem-solving capabilities. When it comes to code revising, ChatGPT can offer suggestions for refactoring code, improving its readability, and optimizing performance [36].

Through Socratic interactions, ChatGPT can encourage students to think critically about code structure, offering advice on how to make code more efficient and scalable. It also provides alternative solutions and highlights areas where

refactoring could improve performance, helping students refine their coding practices. This feedback is invaluable in teaching the importance of writing clean, maintainable code and encourages students to continually revise and improve their work. Furthermore, ChatGPT contributes to the development of critical thinking skills by prompting students to consider different approaches to solving problems, offering alternative solutions, and discussing the trade-offs of each option [37]. This collaborative interaction fosters a mindset of inquiry and reflection, which is essential for cultivating problem-solving abilities and complex decision-making skills. It guides students through logical reasoning and evaluating the pros and cons of different approaches, thus improving their ability to analyze and assess situations from multiple perspectives. Through these interactions, students develop a deeper understanding of not only the "how" but also the "why" behind their decisions, which is crucial in fostering a higher level of critical thinking in programming education.

GenAI models offer opportunities to enhance critical thinking by engaging students in activities that require higher-order cognitive processes. For example, GenAI can support debugging by providing explanations for why specific errors occur, thus prompting students to evaluate code critically and reflect on their thought processes [38]. Furthermore, using AI tools to analyze multiple problem-solving methods fosters metacognition, an essential component of critical thinking [38]. These iterative, reflective practices align with educational frameworks that emphasize active learning and problem-based learning in computer science.

Incorporating GenAI into computer science education necessitates deliberate pedagogical strategies to ensure that its use enhances learning outcomes rather than fostering dependency. One effective approach involves integrating GenAI into project-based coursework, where students can use these models as assistants to prototype solutions, debug programs, and refine their designs [39]. Similarly, Boguslawski et al. [40] underscores the importance of framing AI tools as cognitive scaffolds—resources that support learning rather than replace the intellectual effort required for problem-solving.

Another promising approach is to teach students to critically evaluate AI-generated outputs. This approach aligns with the goal of fostering computational thinking skills, as students must learn to discern between valid and flawed AI-generated solutions. For example, instructors can design exercises that require students to compare GenAI-generated code with manual solutions, analyze discrepancies, and justify their choices. As highlighted by Cubillos et al. [36], such practices not only improve technical skills but also cultivate a more nuanced understanding of the limitations and ethical implications of AI.

Finally, embedding discussions of AI ethics and bias into the curriculum is essential. GenAI tools can generate biased or inaccurate outputs due to the nature of their training data. Encouraging students to reflect on these limitations fosters a critical mindset and prepares them to use these tools responsibly. According to the ACM Education Advisory Committee [33], incorporating case studies and real-world examples of AI-related failures can further solidify these lessons.

III. Methodology

A. Survey Design

The research in this paper employed a survey with both qualitative and quantitative questions to explore ChatGPT's impact on students' perceptions of their critical thinking skills and abilities. The survey questions for this study were developed based on the six key skill and knowledge dimensions of critical thinking: interpretation, analysis, evaluation, inference, explanation, and self-regulation [41]. The survey included 2 to 4 questions for each critical thinking skill dimension, resulting in a total of 21 questions. These questions comprised a mix of multiple-choice questions, with responses using a Likert scale from 1 (Not at all) to 5 (Extremely), and short-answer questions. A preliminary draft of the survey was piloted with seven students who assessed the clarity of the questions using a 1-3 scale: 1: Not clear and understandable, 2: Somewhat clear and understandable, 3: Clear and understandable. From the test group, 98% of the questions received a "3", with 2% giving feedback a "2", specifically related to simplifying and rewording the questions. The survey was anonymous and did not include any private information to encourage students to provide honest answers. The only demographic information collected was about their class standing and program.

B. Participants

A total of 56 responses were collected, with the participants representing a range of academic levels. First-year students comprised 51% of the respondents, while 27.6% were in their second year. Those in their third year accounted for 12.1% of the sample, and 8.6% were in their fourth year of study. The final survey was distributed via email or given to students in programming classes. The participants were not offered any incentives, and participation was voluntary. Participants represented various majors, including Cyber Security Analytics (37.5%), Security Risk Analysis (5.3%), Information Technology (21.4%), and Computer Science (30.3%), and other majors (4.5%).

C. Results of Likert-Scale Questions

Table I presents the mean and standard deviation values for each question, which are coded according to their critical thinking dimensions. Table II provides the averages and standard deviations of the questions within each critical thinking dimension.

DOI: 10.62915/2472-2707.1220

TABLE I. LIKERT SCALE QUESTIONS FOR INTERPRETATION (IT), ANALYSIS (A), EVALUATION (E), INFERENCE (IN), EXPLANATION (EX), AND SELF-REGULATION (SR). (N=56)

Question No	Question	Mean	Standard Deviation
IT1	To what extent has Chat GPT influenced your ability to comprehend complex programming concepts or code snippets?	2.61	0.985
IT2	Do you believe Chat GPT has contributed to your understanding of programming language syntax?	2.71	1.004
A1	To what extent has Chat GPT assisted you in breaking down complex programming problems into smaller, more manageable components for analysis?	2.66	1.066
A2	Have you seen any improvement in your ability to identify patterns or potential code errors through Chat GPT?	2.59	1.092
A3	Has the analysis of AI-generated suggestions influenced your approach to debugging and optimizing code in your programming projects?	2.64	1.167
E1	Has Chat GPT influenced your capacity to assess the efficiency of different programming solutions or approaches?	2.43	1.142
E2	How frequently do you critically evaluate the relevance and reliability of programming codes provided by Chat GPT?	3.20	1.420
E3	Do you believe Chat GPT has contributed to your ability to judge the strengths and weaknesses of various programming methodologies or paradigms?	2.27	1.036
IN1	Has Chat GPT enhanced your ability to make logical inferences when faced with programming-related uncertainties or ambiguities?	2.54	1.061
IN2	Do you think Chat GPT has supported your skills in making educated guesses based on incomplete code?	2.52	1.128
IN3	How often do you experience any difficulties in making accurate inferences from AI-generated suggestions?	2.52	1.112
EX1	Has the process of articulating your thoughts and explaining programming concepts to Chat GPT impacted your ability to communicate technical information clearly?	2.34	1.164
EX2	Do you believe Chat GPT has contributed to your proficiency in explaining complex programming logic or code structures?	2.29	0.967
SR1	Has the use of Chat GPT influenced your ability to reflect on and assess your own thinking processes while working on programming tasks?	2.41	1.023
SR2	Do you believe Chat GPT has contributed to your self-awareness and adaptability in approaching different programming challenges?	2.45	1.127
SR3	Has Chat GPT supported your efforts in setting goals and managing your time effectively during programming projects?	2.32	1.162

The survey results provided insight into how ChatGPT affects students' critical thinking and programming skills. While the tool offers moderate support in areas like problem comprehension, problem-solving, and logical inference, its influence on deeper critical evaluation, communication, and self-reflection is more limited and inconsistent. Students generally find ChatGPT helpful in understanding complex programming concepts and language syntax, particularly in breaking down tasks into manageable parts. However, ChatGPT only dramatically enhances comprehension for some students, indicating that while it is useful for providing explanations, it struggles to adapt to different levels of understanding. When it comes to critical evaluation, there is significant variability. Some students benefit from using ChatGPT to assess the efficiency of solutions, while others accept its output with less scrutiny. This inconsistency raises a concern that students may not always critically evaluate the relevance or accuracy of AI-generated code, potentially undermining their motivation to engage deeply with the material.

ChatGPT's impact on inference skills appears to be limited. Students report slight improvements in their ability to explain programming logic or articulate technical information. This may stem from ChatGPT's focus on providing answers instead of prompting students to verbalize their thought processes. Consequently, students may not acquire the necessary practice to explain complex concepts, which is vital for problem-solving and collaboration. Regarding self-reflection, ChatGPT offers only moderate support. Although students recognize some influence on their ability to reflect on their problem-solving processes, ChatGPT does not significantly enhance goal setting, time management, or metacognitive skills. This suggests that ChatGPT, in its current form, encourages reactive problem-solving rather than fostering proactive planning or reflection necessary for long-term skill development. ChatGPT shows some utility in debugging and making inferences from incomplete information, although its effectiveness in these areas varies. While ChatGPT helps students navigate uncertainties, its suggestions may not always align perfectly with the specific context, leading to mixed outcomes. Overall, ChatGPT serves as a useful supplementary tool, particularly for immediate problem-solving; however, it does not fully support deeper critical thinking, effective communication, or self-reflection. To maximize its educational benefits, it should be integrated into broader pedagogical strategies that emphasize critical evaluation, reflection, and communication, encouraging students to critique AI outputs and engage more deliberately with their learning processes.

As shown in Table II, all Cronbach's Alpha values were higher than 0.70, indicating the internal consistency of the responses in each dimension. The results indicate moderate levels of student engagement with ChatGPT across various critical thinking components. The Interpretation (IT) dimension has a mean of 2.66 and a standard deviation of 0.92, suggesting that students generally find ChatGPT moderately helpful in understanding and interpreting programming concepts. However, there is some variation in responses. For Analysis (A),

the mean is 2.63, with a slightly higher standard deviation of 1.00, indicating a moderate influence of ChatGPT on students' ability to break down and analyze programming problems. The Alpha value of 0.844 also suggests good reliability. The Evaluation (E) dimension has a mean value of 2.63 and a standard deviation of 0.97, indicating that students are somewhat critical in assessing the relevance and accuracy of ChatGPT's outputs. For Inference (IN), the mean of 2.52 and standard deviation of 0.86 suggest a moderate ability to make logical inferences based on incomplete information when using ChatGPT. The lower Alpha value of 0.725 implies somewhat less reliable internal consistency for this dimension. Explanation (EX) has the lowest mean value of 2.31 and a standard deviation of 1.00, suggesting that ChatGPT is perceived as less effective in enhancing students' ability to articulate programming logic or communicate technical information clearly. Finally, the Self-Regulation (SR) dimension, with a mean of 2.39 and a standard deviation of 0.95, indicates that ChatGPT offers moderate support for self-reflective practices, such as goal setting and time management. Overall, the results suggest that while ChatGPT provides moderate support across interpretation, analysis, and evaluation, its impact on more advanced cognitive processes, such as inference, explanation, and self-regulation, is lower.

TABLE II. THE MEAN AND STANDARD DEVIATION OF THE CRITICAL THING DIMENSIONS ALONG WITH THE CRONBACH ALPHA VALUES. (N=56)

Dimensions of Critical Thinking	Mean	Standard Deviation	Cronbach's Alpha
INTERPRETATION (IT)	2.66	0.92	0.850
ANALYSIS (A)	2.63	1.00	0.844
EVALUATION (E)	2.63	0.97	0.888
INFERENCE (IN)	2.52	0.86	0.725
EXPLANATION (EX)	2.31	1.00	0.850
SELFREGULATION (SR)	2.39	0.95	0.822

As shown in Table II, all Cronbach's Alpha values were higher than 0.70, indicating the internal consistency of the responses in each dimension. The results indicate moderate levels of student engagement with ChatGPT across various critical thinking components. The Interpretation (IT) dimension has a mean of 2.66 and a standard deviation of 0.92, suggesting that students generally find ChatGPT moderately helpful in understanding and interpreting programming concepts. However, there is some variation in responses. For Analysis (A), the mean is 2.63, with a slightly higher standard deviation of 1.00, indicating a moderate influence of ChatGPT on students' ability to break down and analyze programming problems. The Alpha value of 0.844 also suggests good reliability. The Evaluation (E) dimension has a mean value of 2.63 and a standard deviation of 0.97, indicating that students are somewhat critical in assessing the relevance and accuracy of ChatGPT's outputs. For Inference (IN), the mean of 2.52 and standard deviation of 0.86 suggest a moderate ability to make logical inferences based on incomplete information when using ChatGPT. The lower Alpha value of 0.725 implies somewhat less reliable internal consistency for this dimension. Explanation (EX) has the lowest mean value of 2.31 and a standard deviation of 1.00, suggesting that ChatGPT is perceived as less effective in enhancing students' ability to articulate programming logic or communicate technical information clearly. Finally, the Self-Regulation (SR) dimension, with a mean of 2.39 and a standard deviation of 0.95, suggests that ChatGPT provides moderate support for self-reflective practices, including goal setting and time management. Overall, the results indicate that while ChatGPT provides moderate support across interpretation, analysis, and evaluation, its impact on more advanced cognitive processes, such as inference, explanation, and self-regulation, is lower.

D. Results of Qualitative Analysis

Open-ended responses were analyzed through a process of data coding and categorization to identify recurring themes, including the impact on critical thinking, conceptual understanding, practical problem-solving, and challenges encountered. A thematic analysis was conducted to identify patterns within the qualitative data and understand the diverse perspectives on how ChatGPT impacted users' learning processes and problem-solving strategies. Finally, sentiment analysis was performed by categorizing the responses as Positive, Mixed, and Negative. This analysis approach provided a structured framework for interpreting and presenting the qualitative data. The results of this analysis are given in Tables III to VII. Since not all participants responded to the open-ended questions, the category percentages in the tables were calculated based on the number of responses in each category relative to the overall number of respondents to the questions, as indicated in the captions of the figures.

Based on the qualitative analysis in Table III, a considerable percentage of respondents utilized ChatGPT for coding assistance, debugging, conceptual understanding, project implementation, and general support. Specifically, 65% of respondents reported using ChatGPT for writing or debugging code, with 35% noting its effectiveness in clarifying syntax issues, especially in languages like C++, Python, SQL, and JavaScript. Moreover, 45% of participants highlighted ChatGPT's role in identifying and fixing bugs, while 20% found it helpful in explaining errors and preventing future mistakes. In terms of conceptual understanding, 30% utilized ChatGPT to grasp complex programming concepts, such as recursion and SQL datatypes. In comparison, 25% stated it assists in specific tasks like string reversal or finding the highest number in a list. Additionally, 20% of respondents emphasized ChatGPT's support in project implementation, particularly during periods of minimal instruction, while 10% found it beneficial for understanding project requirements and visualizing examples. Furthermore, 15% of participants used ChatGPT as a starting point for overcoming writer's block, and 10% relied on it to resolve specific coding problems when other resources were less effective. The overall sentiment regarding ChatGPT's impact on clarifying ambiguous programming requirements is predominantly positive, with 75% of respondents acknowledging its helpfulness for coding-related tasks, particularly for debugging and understanding complex concepts. Examples of Positive sentiments from the students found ChatGPT "helpful for coding-related tasks, particularly for debugging and understanding complex concepts". Negative sentiments, however, stated that students "used it to get unstuck

DOI: 10.62915/2472-2707.1220

on specific coding problems when other resources were less effective."

A subset of respondents (20%) expressed a neutral or mixed sentiment, suggesting that while ChatGPT aids, additional verification and understanding may be required. Only 5% of participants expressed a negative sentiment, indicating that ChatGPT was not helpful for their programming needs. Furthermore, specific instances provided by respondents illustrate ChatGPT's effectiveness in debugging syntax errors, providing conceptual clarifications, and offering guidance in project implementation. For instance, participants mentioned that ChatGPT helped them debug and fix syntax mistakes, explained complex data types and functions, and provided editable code snippets and guidance during project development. In conclusion, the survey findings highlight the significant impact of ChatGPT in clarifying ambiguous programming requirements, with most respondents acknowledging its effectiveness in various aspects of programming education.

TABLE III. ANALYSIS OF THE QUESTION: "CAN YOU PROVIDE A SHORT EXAMPLE OF AN INSTANCE WHERE CHAT GPT HAS HELPED YOU CLARIFY AMBIGUOUS PROGRAMMING REQUIREMENTS OR SPECIFICATIONS, ENHANCING YOUR INTERPRETATIVE SKILLS?" (N=52)

Thematic Category	**Percentage**	**Description**
Coding Assistance	65%	Used ChatGPT to Write/Debug
	35%	Noted it helped in clarifying syntax issues, particularly in C++, Python, SQL, and JavaScript.
Debugging	45%	Mentioned ChatGPT's role in identifying and fixing
	20%	Found it helpful in explaining errors and preventing future mistakes.
Conceptual Understanding	30%	Used ChatGPT to understand complex programming concepts, such as recursion and SQL datatypes.
	25%	Stated it helped them with specific tasks like reversing a string or finding the highest number in a list.
Project Implementation	20%	Highlighted ChatGPT's assistance in implementing projects, especially during periods of minimal instruction (e.g., virtual instruction weeks).
	10%	Found it beneficial for understanding project requirements and visualizing examples.
General Support	15%	Used ChatGPT as a starting point for overcoming writer's block in coding
	10%	Used it to get unstuck on specific coding problems when other resources were less effective.
Overall Sentiment		
Positive	75%	Found ChatGPT helpful for coding-related tasks, particularly for debugging and understanding complex concepts.
Mixed	20%	Used it to get unstuck on specific coding problems when other resources were less effective.
Negative	5%	Used it to get unstuck on specific coding problems when other resources were less effective.
Specified Instances from Participants		**Sample Responses**
Debugging & Syntax		"Helped me debug and fix syntax mistakes in C++ and Python."
		"Explained a syntax error in my code, allowing me to fix it and avoid similar issues in the future."
Conceptual Clarification		"Explained the DATETIME datatype in MySQL."
		"Clarified how to use certain functions in SQL and Python."
Project Guidance		"Provided editable code snippets and guidance during a virtual instruction period for a web programming class."
		"Assisted in understanding and implementing a Java project by breaking down the problem into manageable parts."

The analysis results in Table IV indicate that a significant portion of respondents acknowledge ChatGPT's role in facilitating comprehension and skill development across various aspects of algorithmic understanding and application. Fifty percent of respondents reported that ChatGPT supports breaking down and explaining code, contributing to a clearer understanding of complex programming concepts. Additionally, 25% found it beneficial for grasping smaller portions of code within larger contexts, highlighting its versatility in addressing different levels of complexity. In terms of algorithm and data structure insight, 20% of respondents mentioned that ChatGPT assists in understanding and practicing these fundamental concepts, while 10% noted its effectiveness in clarifying specific algorithmic principles. This suggests that ChatGPT serves as a valuable tool for both broad conceptual understanding and detailed clarification of algorithmic details. Furthermore, ChatGPT demonstrates utility in error identification and correction, with 30% of respondents using it to identify and fix errors in their code. This result is similar to the observations in Table III. Another 15% appreciated its step-by-step breakdowns for solving coding problems, indicating its value in error resolution and problem-solving. Regarding skill development, 20% of respondents reported that using ChatGPT enhanced their problem-solving skills and ability to analyze code. An additional 10% found it conducive to improving efficiency and focus by reducing busy work, underscoring its role in fostering

productive programming practices. The overall sentiment towards ChatGPT's impact on dissecting and understanding algorithms and data structures is mainly positive, with 60% of respondents acknowledging its effectiveness in explaining and breaking down complex concepts. However, 30% perceive it as more suitable for small tasks rather than deep understanding, and 10% express negative sentiments, preferring alternative resources or human assistance for this purpose. In conclusion, ChatGPT emerges as a valuable tool for improving users' skills in dissecting and understanding algorithms and data structures.

TABLE IV. ANALYSIS OF THE QUESTION "IN WHAT WAYS DO YOU THINK CHAT GPT HAS IMPACTED YOUR SKILL IN DISSECTING AND UNDERSTANDING ALGORITHMS OR DATA STRUCTURES USED IN PROGRAMMING?" (N=49)

Category	Percentage	Description
Understanding And Explanation	50%	Reported that ChatGPT helps in breaking down and explaining code, making it more digestible.
	25%	Found it useful for understanding small portions of code within larger contexts.
Algorithm And Data Structure Insight	20%	Mentioned that ChatGPT helps in understanding and practicing algorithms and data structures.
	10%	Noted that ChatGPT clarifies and explains specific concepts related to algorithms and data structures.
Error Identification and Correction	30%	Used ChatGPT for identifying and fixing errors in their code, which helped them understand the underlying issues better.
	15%	Appreciated the step-by-step breakdowns provided by ChatGPT for solving coding problems.
Skill Development	20%	Stated that using ChatGPT improved their problem-solving skills and their ability to dissect code.
	10%	Felt that it made them more efficient by reducing busy work and allowing them to focus on understanding the code.
Overall Sentiment		
Positive	60%	Respondents found ChatGPT helpful for understanding algorithms and data structures, particularly in explaining and breaking down complex concepts.
Mixed	30%	Respondents felt ChatGPT was useful for small tasks but not significantly impactful for deep understanding.
Negative	10%	Respondents did not find ChatGPT helpful for this purpose, preferring other resources or human assistance.
Specified Instances from Participants		**Responses**
Code Explanation & Breakdown		"ChatGPT segments code in a way that is easier to understand and more digestible."
		"Showed what each component meant and how to put multiple components together."
Algorithm And Data Structure Understanding		"Helped me learn new ways to solve programming problems with easy-to-follow breakdowns."
		"Explained different parts of algorithms, making it quicker to pick up various concepts."
Error Correction		"Helped me find errors in my code and learn from them for future reference."
		"Provided live feedback on mistakes in my code and explained confusing parts."

The analysis in Table V revealed the relationship between users and ChatGPT's suggestions in programming tasks, emphasizing the importance of critical evaluation. The findings showed that while ChatGPT was widely used for programming tasks, users approached its outputs with caution. A majority (70%) consistently check for errors and ensure relevance, with 50% reporting incorrect or non-functional code and 10% encountering significantly flawed outputs. This highlights the importance of user expertise in identifying and correcting mistakes. A quarter (25%) of participants need more trust in ChatGPT, verifying outputs through alternative resources, and 10% refrain from relying on it for advanced tasks. Despite these limitations, 20% of participants value ChatGPT as a foundation for refinement. Overall, ChatGPT serves as a helpful aid; however, its reliability relies on user oversight and critical evaluation.

While a significant portion of respondents, making up 60% (positive sentiment), appreciate ChatGPT's assistance, they exercise caution by thoroughly evaluating its suggestions to ensure reliability. This indicates a positive sentiment toward ChatGPT's potential in aiding programming tasks, tempered by a pragmatic approach to error detection and correction. However, a notable portion, consisting of 30% of respondents, takes a more cautious stance, acknowledging ChatGPT's utility but often requiring additional verification or corrections. This group emphasizes the need for balanced reliance on AI-generated content, recognizing its benefits while remaining alert to potential inaccuracies. Conversely, 10% of respondents expressed outright distrust in ChatGPT's suggestions for critical tasks, opting instead for alternative resources.

Overall, the findings highlight the dynamics of utilizing GenAI in programming tasks. The results emphasize the

importance of users critically evaluating ChatGPT's suggestions to maximize its benefits effectively. This summary of user experiences provides key insights into the evolving role of AI assistance in programming. Many participants rely on ChatGPT's suggestions for programming tasks, yet they often assess its usefulness and reliability with a critical eye. While ChatGPT serves as a helpful starting point and can be advantageous for generating initial code or grasping concepts, its suggestions typically require verification and refinement. Most users do not fully trust ChatGPT's output without further verification, ensuring they understand and validate the code before implementation.

TABLE V. ANALYSIS OF THE QUESTION "CAN YOU SHARE INSTANCES WHERE YOU HAVE CRITICALLY EVALUATED THE RELEVANCE AND RELIABILITY OF SUGGESTIONS PROVIDED BY CHAT GPT IN THE CONTEXT OF YOUR PROGRAMMING TASKS?" (N=43)

Category	Percentage	Description
Checking for Errors and Relevance	70%	Reported consistently checking ChatGPT's code for errors or inaccuracies.
	15%	Mentioned always ensuring the code fulfills their specific requirements before implementation.
Identifying Incorrect Suggestions	50%	Found instances where ChatGPT provided incorrect or non-functional code
	10%	Experienced situations where ChatGPT's suggestions were completely off, requiring significant corrections
Trust and Verification	25%	Stated that they do not fully trust ChatGPT's suggestions and always verify them through other means or resources
	10%	Rarely use ChatGPT for critical or advanced programming tasks due to reliability concerns.
Benefits Despite Errors	20%	Acknowledged that despite occasional errors, the overall benefits of using ChatGPT outweigh the downsides
	10%	Found ChatGPT's suggestions helpful for getting a base code, which they then refine and improve.
Instances of Critical Evaluation	10%	Reported comparing ChatGPT-generated code with their own to ensure accuracy and functionality.
	5%	Mentioned using other resources to cross-check methods or suggestions provided by ChatGPT.
Overall Sentiment		
Positive	60%	Appreciate ChatGPT's assistance but always critically evaluate its suggestions to ensure reliability.
Mixed	30%	Find ChatGPT useful but often requires additional verification or corrections.
Negative	10%	Do not trust ChatGPT's suggestions for critical tasks and prefer other resources.
Specified Instances From Participants		**Responses**
Code Verification		"When the code doesn't look right or the suggestion seems off, I double-check it."
		"Sometimes, ChatGPT returns the exact thing I inputted and says it fixed it, so I always check."
Inaccurate Suggestions		"Asked ChatGPT to write a sample code for an assignment and noticed it had flaws, like turning a 5 into a 2."
		"Requested a specific method and researched it to ensure it exists and does what I need."
Comparative Analysis		"Compared my code with ChatGPT's to see if it was the same and found differences."
		"Asked ChatGPT for code efficiency improvements and then timed the output to verify."
General Use		"I never implement suggestions without understanding them first and checking for potential errors."
		"Used ChatGPT to get a base code, then adjusted it to make it less computer-written and more efficient."

The research findings in Table VI underscore the significant role of explaining concepts to ChatGPT in enhancing users' understanding of programming principles. 15% of respondents reported that articulating questions or code to ChatGPT resulted in clearer comprehension, as it necessitated the precise expression of their thoughts. Another 10% found that providing explanations to ChatGPT helped them realize they already knew the answers or clarified their understanding. Additionally, 15% appreciated ChatGPT's ability to simplify intricate programming concepts using easy-to-understand analogies, and 5% specifically used ChatGPT to "dumb down" complex problems for better understanding. Participants also benefited from the technical terms employed by ChatGPT, with 10% noting that explanations using these terms solidified their understanding, and 5% found that ChatGPT clarified relationships between different components in code, such as objects and instances in Java. Moreover, 20% acknowledged that despite occasional errors, the overall benefits of using ChatGPT outweighed the downsides, with 10% finding it helpful for generating base code to refine and improve. Practical

applications and code examples provided by ChatGPT were instrumental in enhancing users' understanding, with 10% citing improvements during internships or projects, such as code conversion or website building, and 5% noting that asking for further examples helped solidify their grasp of concepts. However, the identified themes and relatively low percentages do not support a strong, deep understanding of the concepts. In fact, these percentages are relatively low compared to those related to debugging or error correction observed in earlier questions.

In terms of overall sentiment, 60% of respondents expressed a positive view, finding that explaining concepts to ChatGPT led to a deeper understanding and clarification of programming concepts. In comparison, 25% had mixed experiences, and 15% did not find explaining concepts to ChatGPT helpful at all. These findings emphasize ChatGPT's value in facilitating both broad conceptual understanding and specific problem-solving in programming. Despite these positive outcomes, a segment of respondents expressed neutral or negative sentiments, suggesting that while explaining concepts to ChatGPT may offer benefits, it may not be the sole or preferred method for all users to deepen their understanding of programming. Overall, the findings highlight the potential of explaining concepts to ChatGPT as a supplementary learning tool in programming education, offering opportunities for users to articulate their thoughts, simplify complex ideas, and gain practical insights. However, further research is needed to explore the optimal integration of ChatGPT into programming pedagogy and address the concerns raised by users who require assistance in finding a beneficial approach.

TABLE VI. ANALYSIS OF THE QUESTION "CAN YOU SHARE INSTANCES WHERE PROVIDING EXPLANATIONS TO CHAT GPT HAS LED TO A DEEPER UNDERSTANDING OF PROGRAMMING CONCEPTS FOR YOURSELF?" (N=43)

Category	Percentage	Description
Articulating Concepts and Self-Realization	15%	Explaining questions or code to ChatGPT led to better understanding by requiring clear articulation of thoughts.
	10%	Providing explanations to ChatGPT helped users realize they already knew the answers or clarified their understanding.
Simplifying Complex Concepts	15%	Appreciated ChatGPT's ability to simplify complex programming concepts using easy-to-understand analogies.
	5%	Specifically used ChatGPT to "dumb down" complex problems for better understanding.
Understanding Technical Terms and code review	10%	Benefited from ChatGPT explaining code using technical terms, solidifying understanding of programming concepts.
	5%	Found ChatGPT's explanations clarified relationships between different components in code, such as objects and instances in Java.
Benefits Despite Errors	20%	Acknowledged that despite occasional errors, the overall benefits of using ChatGPT outweigh the downsides
	10%	Found ChatGPT's suggestions helpful for getting a base code, which they then refine and improve.
Practical Applications and Code Examples	10%	Experienced improved understanding when using ChatGPT for practical applications, such as code conversion in internships or building websites.
	5%	Asking ChatGPT for further examples or applications of a concept helped grasp the concept better.
Overall Sentiment		
Positive	60%	Found explaining concepts to ChatGPT beneficial, leading to deeper understanding and clarification of programming concepts.
Mixed	25%	Had mixed experiences, finding some benefit but not relying heavily on ChatGPT for deeper understanding.
Negative	15%	Did not find explaining concepts to ChatGPT helpful and did not see significant improvement in their understanding.
Specified Instances from Participants		**Sample Responses**
Java Programming		"ChatGPT helped me understand Java classes and instances better by explaining objects and methods clearly."
		"In my Java course, ChatGPT articulated the intricacies of my code with technical terms, sparking new ideas."
Code Simplification		"I put my code in ChatGPT to simplify it, and it ended up making my data look clean and neat, teaching me better ways to structure my code."
		"When writing scripts with SQL, I used ChatGPT to check if I had done it right, which helped me study for exams."
Conceptual Understanding		"Explaining complex problems to ChatGPT and asking for simplified explanations helped me understand scenarios better."
		"Providing ChatGPT with the problem, what I understand, and what I don't helped pinpoint my flaws and improve my understanding."
Internships and Projects		"In my internship for code conversion, explaining the code's architecture to ChatGPT and visualizing its proposed syntax in another language enhanced my understanding."
		"When building a website, ChatGPT was beneficial in tasks like centering objects and understanding dictionaries."

DOI: 10.62915/2472-2707.1220

The findings on challenges in maintaining focus and discipline while using ChatGPT, along with the self-regulation strategies employed by users, are presented in Table VII. Most respondents (55%) reported no significant challenges in maintaining focus or discipline, indicating a seamless integration of ChatGPT into their workflow. Interestingly, 20% of users view ChatGPT as a complementary tool, indicating a deliberate effort to balance its use alongside traditional problem-solving approaches. Additionally, 10% noted that their limited use of ChatGPT mitigates any potential impact on their focus, implying a strategic approach to its utilization. However, 15% of respondents acknowledged facing challenges, including over-reliance on ChatGPT or succumbing to distractions. This highlights the importance of addressing potential pitfalls associated with AI dependency in programming environments. Regarding self-regulation strategies, users adopt various approaches to mitigate challenges and optimize their workflow. For instance, 25% emphasize using ChatGPT selectively, primarily for specific issues or as a reference tool. Another 10% leverage ChatGPT for quick answers or code validation while preserving critical thinking for projects. Moreover, 10% prefer exploring alternative resources before resorting to ChatGPT. Only 5% of users consciously limit their ChatGPT usage to avoid over-reliance, reflecting self-awareness and proactive management of AI dependency. Similarly, another 5% prioritize logical understanding of code before seeking assistance from ChatGPT, indicating a commitment to fostering independent problem-solving skills. Lastly, 5% utilize ChatGPT to generate ideas or enhance conceptual understanding rather than rely solely on its code-generation capabilities. Overall, the findings show the need for a balanced approach in integrating ChatGPT into programming tasks. While it offers valuable assistance, users must exercise self-regulation and critical thinking to mitigate potential challenges and optimize their learning experience. While most respondents did not face significant challenges in maintaining focus and discipline while using ChatGPT, a small portion acknowledged the potential for over-reliance and distraction.

TABLE VII. THE ANALYSIS OF THE QUESTION "HAVE YOU ENCOUNTERED ANY CHALLENGES IN MAINTAINING FOCUS AND DISCIPLINE WHILE USING CHAT GPT AS A RESOURCE, AND IF YES, HOW HAVE YOU MANAGED SELF-REGULATION IN SUCH SITUATIONS?" (N=32)

Category	**Percentage**	**Description**
Challenge Reported	55%	ChatGPT is not reliable enough for schoolwork and doesn't provide good examples or explanations.
Use ChatGPT as a Complementary Tool	20%	Use ChatGPT for assistance when hitting roadblocks in programming or to check code.
Use ChatGPT to Understand Concepts	10%	ChatGPT doesn't always provide fully correct answers, so it's not used for learning concepts.
Use to save time	15%	No challenges, see ChatGPT as a time-saver.
Limited Use and Complementary Role	25%	ChatGPT is used mainly for checking code or simplifying difficult topics.
Use ChatGPT Primarily for Quick Answers	10%	Use ChatGPT when roadblocks occur in coding but try solving the issue independently first.
Rarely Use ChatGPT, Doesn't Impact Focus	5%	Use ChatGPT only after trying all other resources and attempting independent solutions.
Alternative Resources and Stubbornness	15%	Prefer notes, forums, and multiple approaches before turning to ChatGPT.
Conscious Limitation and Self-Awareness	15%	Limit use to avoid over-reliance, focusing on understanding code before seeking help.
Specified Instances from Participants		**Sample Responses**
Use ChatGPT only for specific issues or as reference		"I use chat gpt only as a way to either check my code but mostly for helping reshare harder topics in a slightly easier way."
Use ChatGPT primarily for quick answers or validation		"If i hit a roadblock in a program i would try and see if i can pass it myself if not i would use chat gpt to program and ask what i missed and how to not miss it again"
Rarely use ChatGPT, so it doesn't impact focus		"I rarely use it, only when I am dead stuck for a long period of time. I will go through all of the notes and every forum I can find to see if anybody had a similar problem and how they went about solving it. Then I try to implement my own way of doing so…."
Consciously decrease ChatGPT usage to avoid over-reliance		"I try not to use it so I can learn to actually code"
Logically understand code before seeking help from ChatGPT		"I know how to code, i just use it to make the code better or more advanced."

IV. IMPLICATIONS OF THE FINDINGS

Our literature survey shows that GenAI has also made significant inroads, particularly in computer science education. Students heavily use GenAI [42, 43] for a variety of programming tasks, including code generation [44-46] and debugging [47, 48]. Instructors adopted LLMs to generate exercises and answers [49-51], develop test cases [52], create personalized questions [51, 52], provide feedback [53-55], and facilitate conceptual understanding [56, 57] and code explanations [51, 56].

The findings suggest that ChatGPT primarily supports interpretation and analysis tasks, but has a weaker impact on

more complex cognitive processes, such as explanation and self-regulation. This disparity suggests that relying solely on ChatGPT as a standalone resource may not be sufficient for developing higher-order critical thinking skills. However, the current research has limited empirical evidence on the use of GenAI on students' critical thinking. The relatively low ratings for explanation highlight ChatGPT's limited ability to enhance students' skills in articulating programming logic. Students can often benefit from being asked to explain why a solution works or how they arrived at a particular conclusion. If ChatGPT is directly used in computing classes, instructors may need to develop more structured assignments or exercises to make sure that students are prompted to reflect on, critique, or justify ChatGPT outputs [27]. Such a structured approach can reinforce rather than replace students' problem-solving processes.

We also observed a modest impact on self-regulation. This finding suggests that GenAI alone may not significantly cultivate critical metacognitive abilities. A prior study reports an insignificant relationship between students' self-regulation strategies and their AI use [58]. Considering the importance of self-regulation in independent learning and long-term skill development, instructors could integrate more self-reflections and periodic check-ins when students use GenAI while performing tasks [59, 60]. For example, self-regulating prompts such as "What strategy should I use to reflect/manage/guide/monitor my behavior in learning …?" [61] can help students develop self-regulation habits.

Our data show that students' reliance on ChatGPT varies widely. While the majority report no challenges and seamless usage, a smaller but notable proportion (15%) of the participants highlight risks of over-reliance or distraction. This finding points to a personalized adoption of ChatGPT. Some students view it as an occasional helper, while others see it as integral to their workflow, and still others remain wary due to concerns about reliability or academic integrity.

Our preliminary findings indicate that students who critically engage with GenAI in a structured manner are more likely to develop stronger critical thinking skills, as they consider all dimensions of critical thinking. GenAI presents an opportunity to enhance learning and critical analysis in computing classes when utilized effectively. Therefore, educators should prioritize developing exercises that encourage students to engage in deep inquiry and iterative refinement of their understanding, instead of merely copying answers from GenAI. Self-reflection should play a crucial role in these exercises.

This study offers valuable insights into how students perceive the impact of ChatGPT on their critical thinking skills; however, the limitations of the research must be acknowledged. First, the study relies on self-reported data, which means students' responses may not wholly represent their actual skills. Second, the sample size consists of students from a single institution, which has a limited scope, making it challenging to apply the findings to other subjects. Third, the study captures only a single point in time, so we cannot determine how students' critical thinking abilities change with the ongoing use of GenAI, especially given its rapidly evolving nature. Furthermore, not all students responded to the open-ended questions, which may have impacted the identified themes. Finally, the research does not compare ChatGPT users to non-users, making it difficult to ascertain whether AI truly enhances critical thinking. Future studies should tackle these concerns by utilizing more extensive and diverse samples, tracking students over time, and including direct assessments of their skills. Additionally, observational studies can be considered for future research.

Conclusion

This paper provides valuable insights into the multifaceted impacts of GenAI tools on critical thinking skills in programming education. By examining both qualitative and quantitative survey data, it highlights differences in how students perceive and interact with GenAI in programming tasks. The quantitative data reveals broad trends and emphasizes a general, moderate favorability toward AI tools for problem-solving and conceptual understanding. Qualitative analyses uncover more detailed insights into challenges, such as over-reliance on GenAI and the importance of guidance in fostering genuine critical thinking. The results show that students tend to use GenAI more for error correction and debugging to gain a deeper understanding of programming concepts. Overall, participants benefit more from the interpretation, analysis, and evaluation dimensions of critical thinking compared to inference, explanation, and self-regulation dimensions. These patterns are observed in both quantitative and qualitative analyses. This indicates the need for carefully designed pedagogical strategies that balance the advantages of GenAI tools with active learning principles that promote critical thinking. While AI can enhance understanding and efficiency, it also risks diminishing student autonomy if not integrated thoughtfully. Therefore, educators must ensure that AI tools complement, rather than replace, essential critical thinking exercises. Future research should further explore contextual factors, such as individual learning styles and course content, to refine strategies for integrating GenAI. Moreover, longitudinal studies could assess the sustained impact of GenAI tools on the development of critical thinking skills and career readiness. The findings also suggest that educators may develop more structured approaches to learning with GenAI, where reflection and self-regulation are utilized more explicitly.

Acknowledgement

This research is partially sponsored by the National Science Foundation (NSF) Grant (DUE-IUSE-2120936) and the Penn State Franco Undergraduate Research Endowment. Any opinions and findings expressed in this material are of the authors and do not necessarily reflect the views of the NSF.

References

[1] T. N. T. Nguyen, N. V. Lai, and Q. Nguyen, "Artificial Intelligence (AI) in Education: A Case Study on ChatGPT's Influence on Student Learning Behaviors," Educational Process: International Jornal, vol. 13, no. 2, pp. 105-121 2024, doi: 10.22521/edupij.2024.132.7.

[2] N. Talagala. (2024) How AI Will (or Should) Change Computer Science Education. Forbes. Available: https://www.forbes.com/sites/nishatalagala/2024/11/30/how-ai-will-or-should-change-computer-science-education/

[3] R. Yilmaz and F. G. K. Yilmaz, "Augmented intelligence in programming learning: Examining student views on the use of ChatGPT for

programming learning," Computers in Human Behavior: Artificial Humans, vol. 1, no. 2, p. 100005, 2023.

[4] C. P. Dwyer, M. J. Hogan, and I. Stewart, "The promotion of critical thinking skills through argument mapping," In: Critical Thinking Editor: Christopher P. Horvath and James M. Forte, 2011.

[5] P. Facione, "Critical thinking: A statement of expert consensus for purposes of educational assessment and instruction. Research Findings and Recommendations.," American Philosophical Association, 1990.

[6] A. Konak, S. Kulturel-Konak, and H. Liu, "Entrepreneurial Mindset & Innovative Thinking Skills," in ASEE Zone 1 Conference-Spring State College, PA, March 30 - April 1 2023, pp. 1-10. [Online]. Available: https://216.185.13.174/45066

[7] C. McLoughlin and M. J. Lee, "Personalised and self regulated learning in the Web 2.0 era: International exemplars of innovative pedagogy using social software," Australasian Journal of Educational Technology, vol. 26, no. 1, 2010.

[8] A. Konak and C. J. S. F. Clarke, "Augmenting Critical Thinking Skills in Programming Education through Leveraging Chat GPT: Analysis of its Opportunities and Consequences," in 2023 ASEE Mid Atlantic Section Fall Conference, College of New Jersey, NJ, October 27-28 2023, pp. 1-10, doi: https://doi.org/10.18260/1-2--45117.

[9] T. Rasul et al., "The role of ChatGPT in higher education: Benefits, challenges, and future research directions," Journal of Applied Learning and Teaching, vol. 6, no. 1, pp. 41-56, 2023.

[10] D. Parsons and P. Haden, "Programming osmosis: Knowledge transfer from imperative to visual programming environments," in Proceedings of The Twentieth Annual NACCQ Conference, 2007: Citeseer, pp. 209-215.

[11] S. Grover and R. Pea, "Computational Thinking in K–12: A Review of the State of the Field," Educational Researcher, vol. 42, no. 1, pp. 38-43, 2013/01/01 2013, doi: 10.3102/0013189X12463051.

[12] R. Scherer, F. Siddiq, and B. Sánchez-Scherer, "Some Evidence on the Cognitive Benefits of Learning to Code," (in English), Frontiers in Psychology, Opinion vol. 12, 2021-September-09 2021, doi: 10.3389/fpsyg.2021.559424.

[13] B. J. Zimmerman, "Becoming a Self-Regulated Learner: An Overview," Theory Into Practice, vol. 41, no. 2, pp. 64-70, 2002/05/01 2002, doi: 10.1207/s15430421tip4102_2.

[14] M. Mukherjee, N. T. Le, Y.-W. Chow, and W. Susilo, "Strategic approaches to cybersecurity learning: A study of educational models and outcomes," Information, vol. 15, no. 2, p. 117, 2024.

[15] Z. Baig and S. Zeadally, "Cyber-security risk assessment framework for critical infrastructures," Intell. Autom. Soft Comput, vol. 25, no. 1, pp. 121-130, 2019.

[16] J.-N. Tioh, M. Mina, and D. W. Jacobson, "Cyber security training a survey of serious games in cyber security," in 2017 IEEE Frontiers in Education Conference (FIE), 2017: IEEE, pp. 1-5.

[17] V. I. Aremu, J. O. Okuntade, and O. E. Ebimomi, "Computer Programming Language as Tool for Developing Primary School Pupils' Academic and Thinking Skills," Ilorin Journal of Education, vol. 42, no. 2, pp. 69-74, 2022.

[18] S. D'mello and A. Graesser, "AutoTutor and affective AutoTutor: Learning by talking with cognitively and emotionally intelligent computers that talk back," ACM Transactions on Interactive Intelligent Systems (TiiS), vol. 2, no. 4, pp. 1-39, 2013.

[19] D. Weintrop et al., "Defining computational thinking for mathematics and science classrooms," Journal of science education and technology, vol. 25, pp. 127-147, 2016.

[20] S. Hennessy and P. Murphy, "The potential for collaborative problem solving in design and technology," International journal of technology and design education, vol. 9, pp. 1-36, 1999.

[21] P. Williams, "Education Evolution: Pedagogical Techniques for the Modern World," International Journal of Research and Review Techniques, vol. 2, no. 2, pp. 7-13, 2023.

[22] I. S. Lytra, "Reinventing Socratic Irony's Educational Character," University of Edinburgh, 2020.

[23] N. Morten, "Metacognition and Reflective Teaching: A Synergistic Approach to Fostering Critical Thinking Skills," Research and Advances in Education, vol. 2, no. 9, pp. 1-14, 09/20 2023.

[24] N. Huse and N.-T. Le, "The formal models for the socratic method," in Advanced Computational Methods for Knowledge Engineering: Proceedings of the 4th International Conference on Computer Science, Applied Mathematics and Applications, ICCSAMA 2016, 2-3 May, 2016, Vienna, Austria, 2016: Springer, pp. 181-193.

[25] E. Y. Chang, "Examining gpt-4: Capabilities, implications and future directions," in The 10th International Conference on Computational Science and Computational Intelligence, 2023.

[26] P. P. Premkumar, Yatigammana, M. R. K. N., & Kannangara, S., "Impact of Generative AI on Critical Thinking Skills in Undergraduates: A Systematic Review," The Journal of Desk Research Review and Analysis – JDRRA, The Library, University of Kelaniya, vol. 1, no. 1, pp. 01-22 2023.

[27] R. Suriano, A. Plebe, A. Acciai, and R. A. Fabio, "Student interaction with ChatGPT can promote complex critical thinking skills," Learning and Instruction, vol. 95, p. 102011, 2025.

[28] S. Nassar, "Future aspects of critical thinking and AI," in Proceedings of WRFASE International Conference, Abu Dhabi, 2019.

[29] M. Muthmainnah, L. Cardoso, Y. A. M. R. Alsbbagh, A. Al Yakin, and E. Apriani, "Advancing Sustainable Learning by Boosting Student Self-regulated Learning and Feedback Through AI-Driven Personalized in EFL Education," in International Conference on Explainable Artificial Intelligence in the Digital Sustainability, 2024: Springer, pp. 36-54.

[30] A. Styve, O. T. Virkki, and U. Naeem, "Developing Critical Thinking Practices Interwoven with Generative AI Usage in an Introductory Programming Course," in 2024 IEEE Global Engineering Education Conference (EDUCON), 2024: IEEE, pp. 01-08.

[31] E. P. Ododo, N. P. Essien, and A. E. Bassey, "Effect of Generative artificial intelligence (AI)-based tool utilization and students' programming self-efficacy and computational thinking skills in JAVA programming course in Nigeria Universities," International Journal of Contemporary Africa Research Network, vol. 2, no. 1, 2024.

[32] C. A. G. d. Silva, F. N. Ramos, R. V. de Moraes, and E. L. d. Santos, "ChatGPT: Challenges and benefits in software programming for higher education," Sustainability, vol. 16, no. 3, p. 1245, 2024.

[33] B. A. Becker et al., "Generative ai in introductory programming," in "Computer Science Curricula 2023, CS2023, Curricular Practices Volume," ACM, 2023. [Online]. Available: https://csed.acm.org/wp-content/uploads/2023/12/Generative-AI-Nov-2023-Version.pdf

[34] N. Hashmi, Z. Li, S. Parise, and G. Shankaranarayanan, "Generative AI's impact on programming students: frustration and confidence across learning styles," Issues in Information Systems, vol. 25, no. 3, 2024.

[35] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[36] C. Cubillos, R. Mellado, D. Cabrera-Paniagua, and E. Urra, "Generative Artificial Intelligence in Computer Programming: Does it enhance learning, motivation, and the learning environment?," IEEE Access, pp. 1-1, 2025, doi: 10.1109/ACCESS.2025.3532883.

[37] I. Roll and R. Wylie, "Evolution and Revolution in Artificial Intelligence in Education," International Journal of Artificial Intelligence in Education, vol. 26, no. 2, pp. 582-599, 2016/06/01 2016, doi: 10.1007/s40593-016-0110-3.

[38] S. Määttä, "Generative artificial intelligence in programming: A scoping review of cognitive aspects," MS, Information Processing Science, University Of Oulu, 2024.

[39] R. Mellado, C. Cubillos, and G. Ahumada, "Effectiveness of Generative Artificial Intelligence in learning programming to higher education students," in 2024 IEEE International Conference on Automation/XXVI Congress of the Chilean Association of Automatic Control (ICA-ACCA), 20-23 Oct. 2024 2024, pp. 1-7, doi: 10.1109/ICA-ACCA62622.2024.10766746.

[40] S. Boguslawski, R. Deer, and M. G. Dawson, "Programming education and learner motivation in the age of generative AI: student and educator perspectives," Information and Learning Sciences, vol. 126, no. 1/2, pp. 91-109, 2025, doi: 10.1108/ILS-10-2023-0163.

[41] C. Firretto and A. Konak. V. Well. (2024). Theoretical Perspectives on Critical Thinking: Implications for Entrepreneurship Education Research and Practice. Available: https://venturewell.org/wp-

content/uploads/Theoretical-Perspectives-on-Critical-Thinking-White-Paper-IUSE.pdf

[42] K. Hanifi, O. Cetin, and C. Yilmaz, "On ChatGPT: Perspectives from Software Engineering Students," in 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS), 22-26 Oct. 2023 2023, pp. 196-205, doi: 10.1109/QRS60937.2023.00028.

[43] R. Ulfsnes, N. B. Moe, V. Stray, and M. Skarpen, "Transforming Software Development with Generative AI: Empirical Insights on Collaboration and Workflow," in Generative AI for Effective Software Development: Springer, 2024, pp. 219-234.

[44] D. Cambaz and X. Zhang, "Use of AI-driven Code Generation Models in Teaching and Learning Programming: a Systematic Literature Review," presented at the Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, Portland, OR, USA, 2024. [Online]. Available: https://doi.org/10.1145/3626252.3630958.

[45] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models," presented at the Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA, 2022. [Online]. Available: https://doi.org/10.1145/3491101.3519665.

[46] S. Prasad, B. Greenman, T. Nelson, and S. Krishnamurthi, "Generating Programs Trivially: Student Use of Large Language Models," presented at the Proceedings of the ACM Conference on Global Computing Education Vol 1, Hyderabad, India, 2023. [Online]. Available: https://doi.org/10.1145/3576882.3617921.

[47] C. Koutcheme, "Training Language Models for Programming Feedback Using Automated Repair Tools," in Artificial Intelligence in Education, Cham, N. Wang, G. Rebolledo-Mendez, N. Matsuda, O. C. Santos, and V. Dimitrova, Eds., 2023// 2023: Springer Nature Switzerland, pp. 830-835.

[48] Q. Ma, H. Shen, K. Koedinger, and S. T. Wu, "How to Teach Programming in the AI Era? Using LLMs as a Teachable Agent for Debugging," in Artificial Intelligence in Education, Cham, A. M. Olney, I.-A. Chounta, Z. Liu, O. C. Santos, and I. I. Bittencourt, Eds., 2024// 2024: Springer Nature Switzerland, pp. 265-279.

[49] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learning and individual differences, vol. 103, p. 102274, 2023.

[50] S. Sarsa, P. Denny, A. Hellas, and J. Leinonen, "Automatic generation of programming exercises and code explanations using large language models," in Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1, 2022, pp. 27-43.

[51] S. MacNeil et al., "Automatically Generating CS Learning Materials with Large Language Models," presented at the Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 2, Toronto ON, Canada, 2023. [Online]. Available: https://doi.org/10.1145/3545947.3569630.

[52] P. Brusilovsky, B. J. Ericson, C. S. Horstmann, C. Servin, F. Vahid, and C. Zilles, "The Future of Computing Education Materials," CS2023: ACM/IEEECS/AAAI Computer Science Curricula, Curricula Practices, 2023.

[53] C. Geng, Y. Zhang, B. Pientka, and X. Si, "Can chatgpt pass an introductory level functional language programming course?," arXiv preprint arXiv:2305.02230, 2023.

[54] H. Nguyen, N. Stott, and V. Allan, "Comparing Feedback from Large Language Models and Instructors: Teaching Computer Science at Scale," presented at the Proceedings of the Eleventh ACM Conference on Learning @ Scale, Atlanta, GA, USA, 2024. [Online]. Available: https://doi.org/10.1145/3657604.3664660.

[55] C. Koutcheme, N. Dainese, S. Sarsa, A. Hellas, J. Leinonen, and P. Denny, "Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4-As-A-Judge," presented at the Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, Milan, Italy, 2024. [Online]. Available: https://doi.org/10.1145/3649217.3653612.

[56] J. Savelka, A. Agarwal, C. Bogart, and M. Sakr, "Large language models (gpt) struggle to answer multiple-choice questions about code," arXiv preprint arXiv:2303.08033, 2023.

[57] B. Qureshi, "ChatGPT in Computer Science Curriculum Assessment: An analysis of Its Successes and Shortcomings," presented at the Proceedings of the 2023 9th International Conference on e-Society, e-Learning and e-Technologies, Portsmouth, United Kingdom, 2023. [Online]. Available: https://doi.org/10.1145/3613944.3613946.

[58] L. E. Margulieux et al., "Self-Regulation, Self-Efficacy, and Fear of Failure Interactions with How Novices Use LLMs to Solve Programming Problems," presented at the Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, Milan, Italy, 2024. [Online]. Available: https://doi.org/10.1145/3649217.3653621.

[59] D. Sun, P. Xu, J. Zhang, R. Liu, and J. Zhang, "How Self-Regulated Learning Is Affected by Feedback Based on Large Language Models: Data-Driven Sustainable Development in Computer Programming Learning," Electronics, vol. 14, no. 1, p. 194, 2025.

[60] P. Prasad and A. Sane, "A Self-Regulated Learning Framework using Generative AI and its Application in CS Educational Intervention Design," in Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, 2024, pp. 1070-1076.

[61] D. Sun, P. Xu, J. Zhang, R. Liu, and J. Zhang, "How Self-Regulated Learning Is Affected by Feedback Based on Large Language Models: Data-Driven Sustainable Development in Computer Programming Learning," Electronics, vol. 14, no. 1, doi: 10.3390/electronics14010194.